

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS



Guía Práctica de Laboratorio

“Sistema de Gestión de Pacientes con LINQ en C#”

“PROGRAMACIÓN II”

DOCENTE:

Ing. Breno Luque Zuñiga

ESTUDIANTE:

Arturo Sebastian Cutipa Flores

TACNA – PERÚ

2026



Guía de Laboratorio N° 02 Unidad 2 “Catálogo de Productos con LINQ en C# ”	3
I. INFORMACIÓN GENERAL	3
Objetivos	3
Equipos, materiales, programas y recursos utilizados	3
II. MARCO TEÓRICO	3
Windows Forms	3
Programación Orientada a Objetos	3
LINQ	3
DataGridView	4
III. PROCEDIMIENTO	4
Paso 1: Crear el proyecto	4
Paso 2: Diseñar la interfaz gráfica	4
Paso 3: Creación de la clase Paciente	4
Paso 4: Declaración de la lista de pacientes	5
Paso 5: Carga de datos de pacientes	5
Paso 6: Implementación de la búsqueda por diagnóstico	5
Paso 7: Implementación del filtro por grupo etario	6
Paso 8: Ejecución y pruebas	7
IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS	8
Visualización de pacientes	8
Búsqueda por diagnóstico	9
Filtro por grupo etario	9
Cambio de filtro	10
Interpretación General	10
V. CUESTIONARIO	10
1. ¿Qué es un evento en Windows Forms?	10
2. ¿Para qué sirve el evento Form1_Load?	10
3. ¿Qué ventaja tiene usar DataGridView frente a otros controles?	10
4. ¿Qué hace el método ToLower() en el código?	11
CONCLUSIONES	11
RECOMENDACIONES	11
BIBLIOGRAFÍA	11
WEBGRAFÍA	11



Guía de Laboratorio N° 02 Unidad 2 “Catálogo de Productos con LINQ en C#”

I. INFORMACIÓN GENERAL

Objetivos

- Desarrollar una aplicación de escritorio en C# utilizando Windows Forms.
- Aplicar programación orientada a objetos mediante la clase Paciente.
- Utilizar colecciones (List<T>) para almacenar registros de pacientes.
- Implementar consultas con LINQ para búsqueda y filtrado de datos.
- Clasificar pacientes por grupos etarios.
- Mostrar información en un DataGridView.

Equipos, materiales, programas y recursos utilizados

- Computadora o laptop.
- Sistema operativo Windows.
- Visual Studio 2019/2022.
- Lenguaje de programación C#.
- Framework .NET Windows Forms.
- Editor de código integrado de Visual Studio.

II. MARCO TEÓRICO

Windows Forms

Es una tecnología de .NET que permite desarrollar aplicaciones de escritorio con interfaz gráfica mediante controles como botones, cajas de texto, etiquetas y tablas.

Programación Orientada a Objetos

Permite representar entidades mediante clases. En este caso se creó la clase Producto con atributos como nombre, precio y categoría.

LINQ

Permite realizar consultas sobre listas de datos de forma sencilla.

Métodos utilizados:

- Where() → Filtrar datos
- ToList() → Convertir resultados
- Contains() → Buscar coincidencias

DataGridView

Control que permite mostrar datos en forma de tabla.

III. PROCEDIMIENTO

Paso 1: Crear el proyecto

Se abrió el entorno de desarrollo **Visual Studio** y se creó un nuevo proyecto de tipo **Windows Forms App (.NET Framework)** con el nombre Ejercicio LINQ2.

Este tipo de proyecto permite desarrollar aplicaciones de escritorio con interfaz gráfica.

Paso 2: Diseñar la interfaz gráfica

Se diseñó el formulario principal (Form1) utilizando controles del cuadro de herramientas.

Se agregaron los siguientes componentes:

Componente	Nombre asignado	Función
TextBox	txtBuscar	Permite ingresar el diagnóstico a buscar
Button	btnBuscar	Ejecuta la búsqueda
DataGridView	dataGridView1	Muestra los registros de pacientes
ListBox	listBox1	Filtra pacientes por grupo etario

Paso 3: Creación de la clase Paciente

Se creó una clase independiente llamada Paciente.cs dentro del proyecto, la cual representa la estructura de los datos de cada paciente.

```
CSharp
public class Paciente
{
    public int Id { get; set; }
    public string Nombre { get; set; }
    public string Apellido { get; set; }
    public int Edad { get; set; }
    public string Diagnostico { get; set; }
}
```

Esta clase permite trabajar con objetos que contienen la información de cada paciente.

Paso 4: Declaración de la lista de pacientes

Dentro del archivo Form1.cs, se declaró una lista para almacenar los registros de pacientes en memoria:

```
CSharp
List<Paciente> pacientes = new List<Paciente>();
```

Paso 5: Carga de datos de pacientes

En el evento Form1_Load, se agregaron los registros de pacientes obtenidos mediante el código QR.

Cada registro fue ingresado manualmente en la lista mediante objetos de tipo Paciente.

```
CSharp
pacientes.Add(new Paciente { Id = 1, Nombre = "Juan", Apellido = "Perez", Edad = 12, Diagnostico = " U07.1 " });
```

Luego se enlazó la lista con el DataGridView:

```
CSharp
dataGridView1.DataSource = pacientes;
```

También se cargaron las categorías en el ListBox:

```
CSharp
listBoxEdad.Items.Add("Todos");
listBoxEdad.Items.Add("0-13");
listBoxEdad.Items.Add("14-18");
listBoxEdad.Items.Add("19-40");
listBoxEdad.Items.Add("41-60");
listBoxEdad.Items.Add("61+");
```

Paso 6: Implementación de la búsqueda por diagnóstico

Se programó el botón **Buscar** para filtrar los pacientes según el diagnóstico ingresado en el TextBox.

```
CSharp
var resultado = pacientes.Where(x =>
```

```
x.Diagnostico.ToLower().Contains(txtBuscar.Text.ToLower()))
.ToList();

dataGridView1.DataSource = resultado;
```

Esto permite mostrar únicamente los pacientes cuyo diagnóstico coincide con el texto ingresado.

Paso 7: Implementación del filtro por grupo etario

Se utilizó el evento `SelectedIndexChanged` del `ListBox` para filtrar los pacientes según su edad.

Se definieron rangos de edad para cada grupo etario:

```
CSharp
switch (grupo)
{
    case "0-13":
        filtro = pacientes.Where(x => x.Edad <= 13).ToList();
        break;

    case "14-18":
        filtro = pacientes.Where(x => x.Edad >= 14 && x.Edad <= 18).ToList();
        break;

    case "19-40":
        filtro = pacientes.Where(x => x.Edad >= 19 && x.Edad <= 40).ToList();
        break;

    case "41-60":
        filtro = pacientes.Where(x => x.Edad >= 41 && x.Edad <= 60).ToList();
        break;

    case "61+":
        filtro = pacientes.Where(x => x.Edad >= 61).ToList();
        break;

    default:
        filtro = pacientes;
        break;
}
```

Los resultados fueron mostrados en el `DataGridView`.



Paso 8: Ejecución y pruebas

Finalmente, se ejecutó la aplicación para verificar su funcionamiento.

Se comprobó:

- La correcta carga de los registros de pacientes
- El funcionamiento de la búsqueda por diagnóstico
- El filtrado por grupo etario
- La visualización adecuada de los datos en el DataGridView

Se observó que el sistema responde correctamente a las acciones del usuario.

IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS

Visualización de pacientes:

	Id	Nombre	Apellido	Edad	Diagnostico
▶	1	Sofia	Castillo	8	B01.9
	2	Mateo	Vargas	45	I10
	3	Valentina	Mendoza	30	U07.1
	4	Sebastian	Castro	22	J11.1
	5	Isabella	Guzman	60	E11.9
	6	Nicolas	Silva	55	U07.1
	7	Camila	Rojas	18	J02.9
	8	Alejandro	Morales	70	I50.9
	9	Daniela	Ortega	27	U07.1
	10	Samuel	Reyes	14	J45.9
	11	Antonia	Fuentes	52	M54.5
	12	Benjamin	Espinoza	33	U07.1
	13	Emilia	Cordova	9	A09.9
	14	Gabriel	Figueroa	48	J44.9
	15	Martina	Pizarro	62	N39.0

Análisis: Se observa que los registros de pacientes fueron cargados correctamente desde la lista hacia el DataGridView, mostrando la información de forma ordenada en columnas (ID, nombre, apellido, edad y diagnóstico).

Búsqueda por diagnóstico:

U07.1		Buscar			
	Id	Nombre	Apellido	Edad	Diagnostico
▶	3	Valentina	Mendoza	30	U07.1
	6	Nicolas	Silva	55	U07.1
	9	Daniela	Ortega	27	U07.1
	12	Benjamin	Espinoza	33	U07.1
	16	Lucas	Navarro	20	U07.1
	21	Florencia	Sanchez	37	U07.1
	24	Victoria	Perez	41	U07.1
	27	Cristobal	Torres	35	U07.1
	30	Elena	Garcia	44	U07.1
	33	Fernando	Fernandez	49	U07.1
	36	Paulina	Nuñez	19	U07.1
	40	Camen	Vega	39	U07.1
	42	Mariana	Peña	23	U07.1
	45	Ignacio	Contreras	77	U07.1
	48	Fransisco	Orellana	38	U07.1

Análisis: El sistema filtra correctamente los pacientes según el diagnóstico ingresado, mostrando únicamente aquellos que coinciden (en este caso los de Covid-19) . Esto demuestra el uso adecuado de consultas LINQ.

Filtro por grupo etario:

		Buscar			
	Id	Nombre	Apellido	Edad	Diagnostico
▶	3	Valentina	Mendoza	30	U07.1
	4	Sebastian	Castro	22	J11.1
	9	Daniela	Ortega	27	U07.1
	12	Benjamin	Espinoza	33	U07.1
	16	Lucas	Navarro	20	U07.1
	17	Trinidad	Araya	40	E11.9
	19	Josefina	Tapia	24	J11.1
	21	Florencia	Sanchez	37	U07.1
	23	Rafael	Gonzalez	29	J02.9
	27	Cristobal	Torres	35	U07.1
	32	Luciana	Rodriguez	31	J11.1
	35	Augusto	Carrasco	26	I10
	36	Paulina	Nuñez	19	U07.1
	40	Camen	Vega	39	U07.1
	42	Mariana	Peña	23	U07.1

Análisis: Se visualizan únicamente los pacientes que pertenecen al grupo etario seleccionado, lo que confirma que el filtrado por edad funciona correctamente.

Cambio de filtro:

Todos
0-13
14-18
19-40
41-60
61+

Buscar

	Id	Nombre	Apellido	Edad	Diagnostico
▶	8	Alejandro	Morales	70	I50.9
	15	Martina	Pizarro	62	N39.0
	18	Maximiliano	Valenzuela	68	I10
	25	Diego	Lopez	73	I50.9
	31	Javier	Martinez	66	J44.9
	34	Catalina	Moran	63	N39.0
	38	Laura	Alvarez	72	M54.5
	43	Felipe	Miranda	67	I50.9
	45	Ignacio	Contreras	77	U07.1
	51	Sofia	Castillo	61	U07.1
	54	Cristian	Aravena	64	E11.9
	58	Patricio	Montoya	69	I10
	62	Mario	Baeza	71	U07.1
	67	Hugo	Guerra	74	I50.9
	76	Cesar	Salazar	62	U07.1

Análisis: El sistema actualiza automáticamente los resultados en el DataGridView, mostrando solo los pacientes que cumplen con el nuevo rango seleccionado.

Interpretación General

Los resultados obtenidos indican que el sistema cumple con los requerimientos planteados, permitiendo gestionar información de pacientes de manera eficiente.

El uso de LINQ facilita la implementación de búsquedas y filtros, mejorando la organización del código y la rapidez en la obtención de resultados.

La interfaz gráfica desarrollada en Windows Forms permite una interacción sencilla y clara para el usuario.

V. CUESTIONARIO**1. ¿Qué es un evento en Windows Forms?**

Es una acción que ocurre en la interfaz (como hacer clic en un botón) y permite ejecutar código asociado.

2. ¿Para qué sirve el evento Form1_Load?

Se ejecuta cuando el formulario se inicia y se utiliza para cargar datos o configurar controles.

3. ¿Qué ventaja tiene usar DataGridView frente a otros controles?



Permite mostrar grandes cantidades de datos en formato tabular de forma ordenada y dinámica.

4. ¿Qué hace el método ToLower() en el código?

Convierte el texto a minúsculas para evitar errores al comparar cadenas en búsquedas.

5. ¿Por qué se usa ToList() en las consultas LINQ?

Porque convierte el resultado de la consulta en una lista que puede ser utilizada por el DataGridView.

CONCLUSIONES

- Se desarrolló correctamente un sistema de gestión de pacientes utilizando Windows Forms.
- Se aplicaron conceptos de programación orientada a objetos mediante la clase Paciente.
- El uso de LINQ permitió realizar búsquedas y filtrados de manera eficiente.
- El sistema permite clasificar pacientes por grupos etarios de forma clara.
- La interfaz gráfica facilita la interacción del usuario con el sistema.

RECOMENDACIONES

- Validar los datos ingresados para evitar errores.
- Agregar funcionalidades como editar o eliminar pacientes.
- Mejorar el diseño visual del formulario.
- Implementar conexión a base de datos para almacenamiento permanente.
- Incluir más filtros para mejorar el análisis de datos.

BIBLIOGRAFÍA

- Deitel, P. C#. Cómo programar. Pearson Educación.
- Microsoft Corporation. Manual de programación en C#.
- Joyanes Aguilar, L. Fundamentos de Programación.

WEBGRAFÍA

- Microsoft Learn. Documentación oficial de C#.
- Microsoft Learn. Guía de LINQ en .NET.
- Microsoft Learn. Windows Forms Documentation.